FINITE AUTOMA

BY Marvin K. Nakayama
For Academic learning

Chapter 1
Regular Languages

Contents

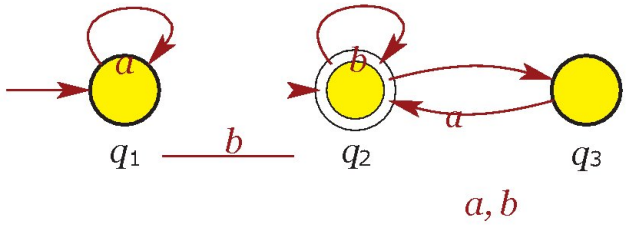
- Finite Automata
- Class of Regular Languages is Closed Under Some Operations
- Nondeterminism
- Regular Expressions
- Nonregular Languages

Introduction

- Now introduce a simple model of a computer having a finite amount of memory.
- This type of machine will be known as a finite-state machine or finite automaton.
- Basic idea how a finite automaton works:
  - It is presented an input string w over an alphabet Σ; i.e., $w \in \Sigma^*$.
  - It reads in the symbols of w from left to right, one at a time.
    After reading the last symbol, it indicates if it accepts or rejects the string.
- These machines are useful for string matching, compilers, etc.

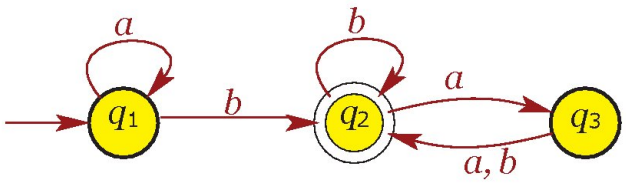
Deterministic Finite Automata (DFA)

Example: DFA with alphabet $\Sigma = \{a, b\}$:

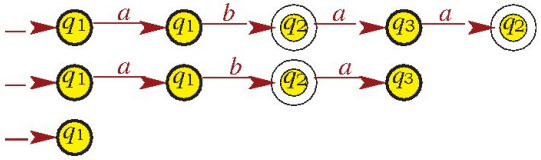


- q_1, q_2, q_3 are the states.
- q_1 is the start state as it has an arrow coming into it from nowhere.
- q_2 is an accept state as it is drawn with a double circle.

Deterministic Finite Automata



- Edges tell how to move when in a state and a symbol from Σ is read.
- DFA is fed input string $w \in \Sigma^*$. After reading last symbol of w,
  - if DFA is in an accept state, then string is accepted
  - otherwise, it is rejected.
- Process the following strings over $\Sigma = \{a, b\}$ on above machine:
  - $abaa$ is accepted
  - aba is rejected
  - ε is rejected



Formal Definition of DFA

Definition: A deterministic finite automaton (DFA) is a 5-tuple

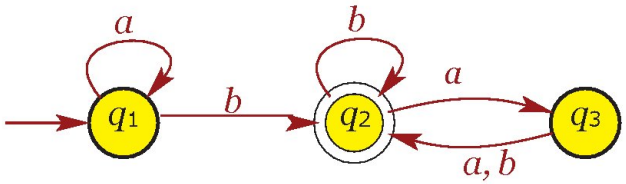
$$M = (Q, \Sigma, \delta, q_0, F),$$

where

1. Q is a finite set of states.
2. Σ is an alphabet, and the DFA processes strings over Σ.
3. $\delta : Q \times \Sigma \to Q$ is the transition function.
   - δ defines label on each edge.
4. $q_0 \in Q$ is the start state (or initial state).
5. $F \subseteq Q$ is the set of accept states (or final states).

Remark: Sometimes refer to DFA as simply a finite automaton (FA).

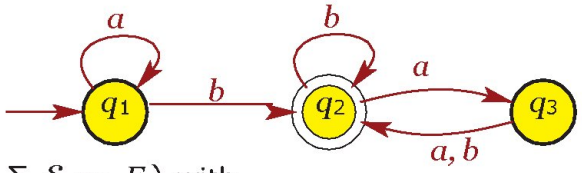
Transition Function of DFA



Transition function $\delta : Q \times \Sigma \to Q$ works as follows:

- For each state and for each symbol of the input alphabet, the function δ tells which (one) state to go to next.
- Specifically, if $r \in Q$ and $\in \Sigma$, then $\delta(r, \ )$ is the state that the DFA goes to when it is in state r and reads in , e.g., $\delta(q_2, a) = q_3$.
- For each pair of state $r \in Q$ and symbol $\in \Sigma$,
  - there is exactly one arc leaving r labeled with .
- Thus, there is no choice in how to process a string.
  - So the machine is deterministic.

Example of DFA



$M = (Q, \Sigma, \delta, q_1, F)$ with

- $Q = \{q_1, q_2, q_3\}$
- $\Sigma = \{a, b\}$
- $\delta : Q \times \Sigma \to Q$ is described as

	a	b
q_1	q_1	q_2
q_2	q_3	q_2
q_3	q_2	q_2

- q_1 is the start state
- $F = \{q_2\}$.

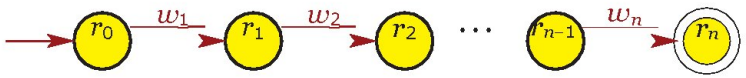
How a DFA Computes

- DFA is presented with an input string $w \in \Sigma^*$.
- DFA begins in the start state.
- DFA reads the string one symbol at a time, starting from the left.
- The symbols read in determine the sequence of states visited.
- Processing ends after the last symbol of w has been read.
- After reading the entire input string
  - if DFA ends in an accept state, then input string w is accepted;
  - otherwise, input string w is rejected.

Formal Definition of DFA Computation

- Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA.
- String $w = w_1 w_2 \ldots\ldots w_n \in \Sigma^*$, where each $w_i \in \Sigma$ and $n \geq 0$.
- Then M accepts w if there exists a sequence of states $r_0, r_1, r_2, \ldots, r_n \in Q$ such that

1. $r_0 = q_0$
   - first state r_0 in the sequence is the start state of DFA;
2. $r_n \in F$
   - last state r_n in the sequence is an accept state;
3. $\delta(r_i, w_{i+1}) = r_{i+1}$ for each $i = 0, 1, 2, \ldots, n-1$
   - sequence of states corresponds to valid transitions for string w.

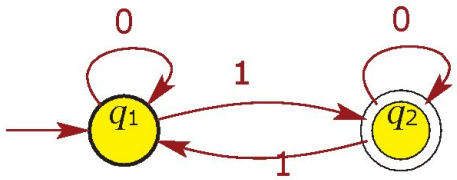


Language of Machine

- Definition: If A is the set of all strings that machine M accepts, then we say
  - $A = L(M)$ is the language of machine M, and
  - M recognizes A.
- If machine M has input alphabet Σ, then $L(M) \subseteq \Sigma^*$.
- Definition: A language is regular if it is recognized by some DFA.

Examples of Deterministic Finite Automata

Example: Consider the following DFA M_1 with alphabet $\Sigma = \{0, 1\}$:



Remarks:

- 010110 is accepted, but 0101 is rejected.
- $L(M_1)$ is the language of strings over Σ in which the total number of 1's is odd.
- Can you come up with a DFA that recognizes the language of strings over Σ having an even number of 1's ?

Summary of Chapter 1

- DFA is a deterministic machine for recognizing certain languages.
- A language is regular if it has a DFA.
- The class of regular languages is closed under union, intersection, concatenation, Kleene-star, complementation.
- NFA can be nondeterministic: allows choice in how to process string.
- Every NFA has an equivalent DFA.
- Regular expression is a way of generating certain languages.
- Kleene's Theorem: Language A has DFA iff A has regular expression.
- Every finite language is regular, but not every regular language is finite.
- Use pumping lemma to prove certain languages are not regular.